Lecture 21

Stacks, Queues and Linked lists

We will cover some basic data structures

- Data structures hold a collection of data (dataset) constructing a certain shape
  - list, tree, table, graph, etc.
- Depending on the types of tasks you will be doing with the dataset, some data structures can be more suitable than the others
- We will start with stacks and queues, which are classic data structures; and they are still commonly used!

Abstract Data Type

- Abstract Data type (ADT) is a type (or class) for objects whose behavior is defined by a set of values and a set of operations
- They are often defined as a class

Stack refers to a stack of data

- A linear data structure with a single access point (i.e., top)
- Behaves like stacks in real world, following the Last-In-First-Out manner

A program has pushed a few items to a stack in the following order: 10, 20, 30, 0, -30. What is the correct sequence of outputs when calling pop() 5 times?

A	10, 20, 30, 0, -30
B	-30, 0, 10, 20, 30
C	30, 10, 20, 0, -30
D	-30, 0, 30, 20, 10

class[Buzz]: stack_pop

Stack ADT offers these operations

- ADT: Abstract Data Type

```
class Stack:
    # Adds the specified object to the top of the stack
    def push(self, obj);

    # Removes the top of the stack and returns it
    def pop (self);

    # Returns the top of the stack without removing it
    def peek(self);
```

```
class Stack:
```

```
class Stack:
  def __init__(self):
    self.stack = []

  def push(self, value):
    self.stack.append(value)

  def pop(self):
    popped = self.peek()
    self.stack = self.stack[:-1]
    return popped

  def peek(self):
    return self.stack[-1]

  def display(self):
    return self.stack
```

You need to consider edge cases!

i.e. when the stack is empty

- pop( )

- peek( )

Queue refers to a queue of data

- Operations
  - Enqueue
  - Dequeue
  - Peek

Queue vs. Stack

Stack	Queue
LIFO	FIFO
Single access point - top	Two access points - front, rear
push( )	enqueue()
pop()	dequeue()
peek()	peek()

Example applications using queues/stacks

- Autograder queuing system
- TextEditor

```
from Stack import Stack

class TextEditor:
  def __init__(self):
    …

def edit(self, text):
  pass

def undo(self):
  pass

def showCurrentText(self):
  pass

def showHistory(self):
  Pass
```

```
from Stack import Stack

class TextEditor:
  def __init__(self):
    self.editHistory = Stack()
    self.currentText = self.editHistory.peek()

def edit(self, text):
  self.editHistory.push(text)
  self.currentText = self.editHistory.peek()

def undo(self):
  self.currentText = self.editHistory.pop()

def showCurrentText(self):
  print (self.editHistory.peek())

def showHistory(self):
  print (self.editHistory.display())
```

Linked Lists

Why would we use LLs over python lists?

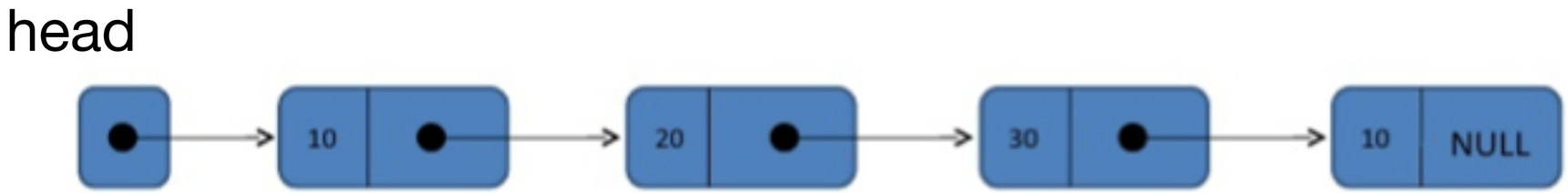
- Python is made out of C
- Python lists are actually built as an array in C
  - array has a fixed length (i.e. static data structure)
  - once it’s initialized you cannot shrink/increase the length unless you declare a new array with a new size and copy over all the values

Adding an element to an array takes time

- Suppose I have an array with 5 elements: [1, 3, 5, 7, 9]
- I want to insert a -1 before 1. What should I do?
  - Create a new bigger array
  - Shift 1, 3, 5, 7, 9 to the right
  - Insert a -1 before 1.
  - Change reference from old array to a new one.

Linked lists can be an alternative

- A linked list is a linear data structure where each element is a separate object
- A linked list is a dynamic data structure
  - The number of nodes in a list is not fixed and can grow and shrink on demand
  - More suitable for any application which has to deal with an unknown number of objects, compared to arrays



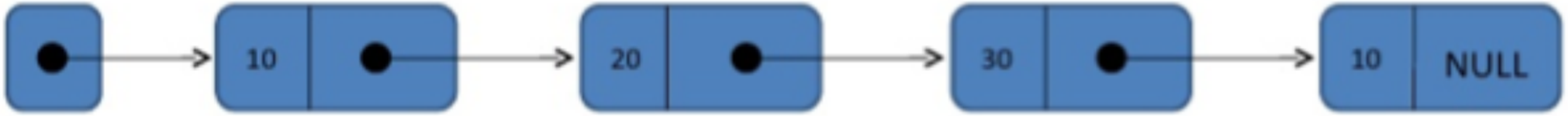
Linked lists have downsides too

- Does not allow direct access to the individual elements
  - If you want to access a particular item then you have to start at the head and follow the references until you get to that item
- Uses more memory compare with an array

A linked List is a chain of data items

- a chain of node: a node inside a node

a linked list




```
class Node:
  …


node = Node(10)
```

Fill in the blank.

```
class Node:
  def __init__(self, value):
    self.item = value
    self.next = __________



# creates a node that has the value 10
node = Node(10)
```

```
class Node:
  def __init__(self, value):
    self.item = value
    self.next = __________



# creates a node that has the value 10
node = Node(10)
node.next = Node(5)
```

Which statement is correct, once `main()` finishes?

```
class Node:
    …

node1 = Node(10)
node2 = Node(5)
node2.next = node1
```

A	node1.item has 10
B	node1.item has 5
C	The reference of node2 points to node1
D	There is a single container that has the actual value and next inside

class[Buzz]: linked_main

Linked List ADT includes these operations

- addFront( )
- addEnd( )
- removeFront( )
- removeEnd( )
- size( )
- display( )
- …

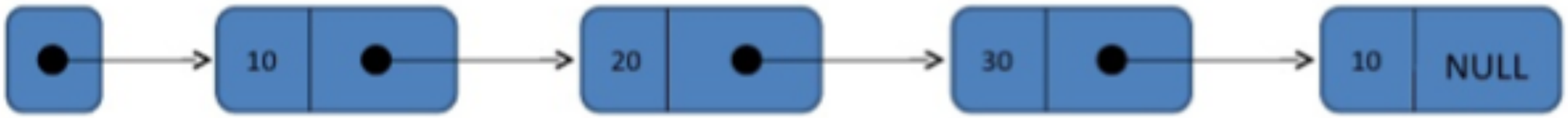
Linked List ADT

```
class Node:
  def __init__(self, value):
    self.value = value
    self.next = None
```

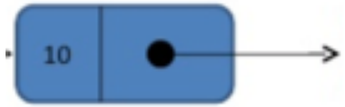
```
class LL:
  def __init__(self):
    self.head = None
    self.size = 0

  def addEnd(self, value): …
  def addFront(self, value): …
  def removeFront(self): …
  def removeEnd(self): …
  def display(self): …
```

AddFront - add to the beginning

head



nodeE

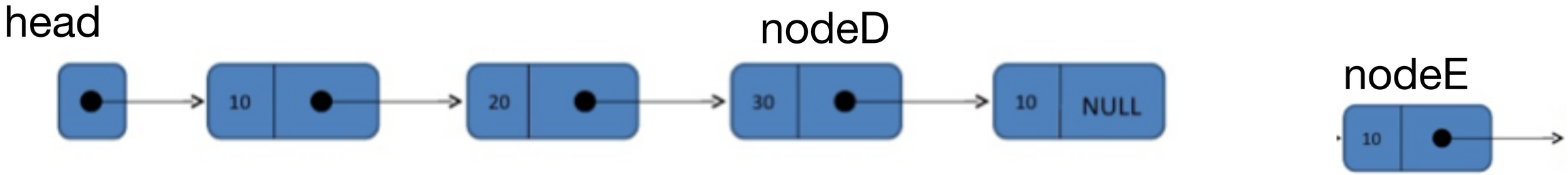


Select a correct implementation of `AddFront`.

A	`head = NodeE;`
B	`nodeE.next = head;`
C	`head = nodeE;` `nodeE.next = head;`
E	`nodeE.next = head;` `head = nodeE;`

class[Buzz]: add_front

AddBack - add to the back



Select a correct implementation of `AddBack`.

A	`nodeD.next = nodeE;`
B	`<loop through the list to get to nodeD first>` `nodeD.next = nodeE;`
C	`nodeC.next.next = nodeE;`
D	`None of the above`

class[Buzz]: add_back